

Plan Testów (Test Plan)

1. Wprowadzenie i cel testowania

Celem testowania jest weryfikacja poprawności działania aplikacji **Employee Manager** oraz jej zgodności z wymaganiami funkcjonalnymi. Testy mają na celu wykrycie niezgodności i błędów w działaniu aplikacji oraz ocenę poprawności interfejsu użytkownika i komunikacji z backendem.

2. Zakres testów

Objęte testami:

- walidacja pól formularza pracownika,
- dodawanie, edycja i usuwanie pracowników,
- pobieranie listy pracowników,
- reset danych,
- obsługa błędów walidacji,
- działanie interfejsu użytkownika,
- testy API,
- testy UI.

Świadomie pominięte:

- testy wydajnościowe,
- testy penetracyjne i szczegółowe testy bezpieczeństwa,
- testowanie wdrożenia produkcyjnego.

3. Strategia testowania

- **Testy funkcjonalne manualne** – sprawdzenie działania formularza oraz walidacji pól z wykorzystaniem technik czarnoskrzynkowych: analizy klas równoważności (EP) i analizy wartości brzegowych (BVA).
- **Testy API** – testy automatyczne z wykorzystaniem Python + pytest oraz requests/http, obejmujące poprawne operacje, błędne dane oraz nieistniejące zasoby.
- **Testy UI** – automatyczne testy interfejsu z wykorzystaniem Python + pytest + Selenium WebDriver.
- **Testy regresji** – ponowne wykonanie wybranych testów po wprowadzeniu zmian w aplikacji.

4. Środowisko testowe

- **System:** Ubuntu 24.04.4 LTS
- **Kernel:** WSL2 — 6.18.33.2-microsoft-standard-WSL2
- **Architektura:** x86_64
- **Przeglądarka:** Google Chrome for Linux
- **Wersja Chrome:** 151.0.7922.71
- **Python:** 3.13+
- **Środowisko Python:** venv
- **Aplikacja:** Employee Manager 1.3.0
- **Adres:** <http://127.0.0.1:8000/>

5. Role i odpowiedzialności

Tester QA:

- analiza wymagań,
- projektowanie i wykonywanie przypadków testowych,
- dokumentowanie wyników,
- zgłaszanie defektów,
- przygotowanie testów API i UI,
- przygotowanie dokumentacji projektowej.

6. Kryteria wejścia i wyjścia

Kryteria wejścia:

- aplikacja jest uruchomiona lokalnie,
- formularz i podstawowe funkcjonalności aplikacji są dostępne,
- środowisko testowe jest skonfigurowane,
- wymagania są dostępne do analizy.

Kryteria wyjścia:

- wykonano zaplanowane przypadki testowe,
- wyniki testów zostały udokumentowane,
- znalezione defekty zostały zgłoszone,
- wykonano wymagane testy automatyczne.

7. Kryteria akceptacji

Aplikacja jest uznawana za spełniającą kryteria akceptacji, jeżeli jej działanie jest zgodne z wymaganiami funkcjonalnymi oraz spełnia założenia określone dla poszczególnych funkcjonalności i przypadków testowych.

8. Ryzyka projektowe i sposoby ich ograniczenia

Ryzyko	Sposób ograniczenia
Niejasne lub sprzeczne wymagania	Analiza wymagań i przygotowanie pytań do Product Ownera
Zmiany w aplikacji podczas testów	Dokumentowanie wersji aplikacji i środowiska testowego
Błędy środowiska lokalnego	Sprawdzenie poprawności uruchomienia aplikacji przed testami
Duża liczba przypadków testowych	Grupowanie testów według funkcjonalności i wykorzystanie technik EP/BVA
Błędy wykryte dopiero podczas automatyzacji	Wykorzystanie wcześniejszych testów manualnych jako podstawy automatyzacji

9. Harmonogram realizacji

Projekt zostanie zrealizowany etapami zgodnie z kolejnością określoną w wymaganiach projektu. Termin oddania projektu przypada **tydzień po zakończeniu szkolenia**.

10. Wykorzystywane narzędzia

- **TestRail** – zarządzanie przypadkami testowymi,
- **JIRA** – zarządzanie defektami,
- **Postman** – manualne testowanie API,
- **Python + pytest + requests/http** – automatyzacja testów API,
- **Python + pytest + Selenium WebDriver** – automatyzacja testów UI,
- **Git + GitHub** – kontrola wersji i przechowywanie kodu,
- **Word/PDF** – dokumentacja projektu.